Creación de Imágenes y Gestión en DockerHub

Unidad 2

Dockerfile • Buenas Prácticas • Build, Tag & Push • DockerHub • Escaneo de Vulnerabilidades

Curso de Contenedores y Orquestación

Agenda

Contenido de la Unidad 2

Crear Imágenes con Dockerfile — 01
Instrucciones, capas y construcción

Buenas Prácticas — 02
Optimización, seguridad y eficiencia

Build, Tag y Push — 03
Flujo completo de construcción y publicación

DockerHub — 04
Registry público, cuenta, repositorios y push

Vulnerabilidades en Imágenes — 05
CVEs, capas inseguras y riesgos comunes

Escaneo de Imágenes — 06
Trivy, Docker Scout y pipelines de seguridad

¿Qué es una Imagen Docker?

Plantilla inmutable de solo lectura para crear contenedores

Una imagen Docker es un paquete ligero, standalone y ejecutable que incluye todo lo necesario para correr una aplicación: código fuente, runtime, librerías del sistema, herramientas y configuración.

Capas (Layers)

Cada instrucción del Dockerfile crea una capa de solo lectura. Se almacenan de forma incremental y se comparten.

Inmutable

Una vez construida no cambia. Los cambios del contenedor van en una capa de escritura temporal.

Tags y Versiones

Cada imagen tiene nombre y tag (ej: nginx:1.25-alpine). El tag por defecto es :latest.

Registry

Se almacenan en registries (DockerHub, ECR, GCR, ACR, Harbor) para distribución global.

Instrucciones del Dockerfile

Cada instrucción crea una nueva capa en la imagen

FROM Imagen base. Primera instrucción obligatoria.

```
FROM python:3.11-slim
```

WORKDIR Establece el directorio de trabajo.

```
WORKDIR /app
```

COPY Copia archivos del host al contenedor.

```
COPY . /app
```

ADD Como COPY pero soporta URLs y tar.

```
ADD app.tar.gz /opt/
```

RUN Ejecuta comandos durante el build.

```
RUN pip install flask
```

ENV Define variables de entorno.

```
ENV APP_ENV=production
```

EXPOSE Documenta el puerto que usa la app.

```
EXPOSE 8080
```

CMD Comando por defecto al iniciar.

```
CMD ["python","app.py"]
```

ENTRYPOINT Ejecutable principal del contenedor.

```
ENTRYPOINT ["nginx"]
```

USER Define el usuario para RUN, CMD, ENTRYPOINT.

```
USER appuser
```

Anatomía de un Dockerfile

Ejemplo completo con anotaciones por sección

Dockerfile

```dockerfile
# 1. Imagen base ligera
FROM node:20-alpine

# 2. Directorio de trabajo
WORKDIR /app

# 3. Dependencias primero (cache)
COPY package*.json ./
RUN npm ci --only=production

# 4. Código fuente
COPY src/ ./src/

# 5. Usuario no-root
RUN adduser -D appuser
USER appuser

# 6. Puerto y comando
EXPOSE 3000
CMD ["node","src/index.js"]
```

Explicación por sección

1. Alpine reduce la imagen de ~1GB a ~180MB
2. WORKDIR crea el directorio si no existe
3. Copiar package.json antes que el código aprovecha el cache de Docker
4. El código cambia frecuentemente; va después para no invalidar cache
5. Nunca correr como root en producción
6. CMD en formato exec (JSON) es preferible a shell form

Sistema de Capas (Layers)

Cada instrucción RUN, COPY, ADD crea una capa de solo lectura

Stack de Capas

← Escritura

```
Container Layer (R/W)
COPY src/ ./src/
RUN npm ci
COPY package*.json ./
WORKDIR /app
node:20-alpine (base)
```

↑ Capas de solo lectura (compartidas entre contenedores)

Beneficios del Sistema de Capas

Cache Inteligente

Si una capa no cambia, Docker la reutiliza. Solo reconstruye desde la capa modificada hacia abajo.

Capas Compartidas

Múltiples imágenes basadas en alpine:3.18 comparten las capas base, ahorrando disco.

Builds Rápidos

Separar dependencias del código permite que 90% del build use cache.

Eficiencia de Red

Al hacer pull/push solo se transfieren las capas que faltan en el destino.

Buenas Prácticas (1/2)

Optimización de tamaño y rendimiento

Usar imágenes base ligeras

✗

```
FROM ubuntu:22.04  (77MB)
```

✓

```
FROM python:3.11-slim  (45MB)
FROM python:3.11-alpine  (17MB)
```

Minimizar número de capas

✗

```
RUN apt update
RUN apt install -y curl
RUN apt install -y git
```

✓

```
RUN apt update && apt install -y \
    curl git && \
    rm -rf /var/lib/apt/lists/*
```

Aprovechar el cache de build

✗

```
COPY . /app
RUN pip install -r requirements.txt
```

✓

```
COPY requirements.txt /app/
RUN pip install -r requirements.txt
COPY . /app
```

Usar .dockerignore

✗

Sin .dockerignore (copia .git, node_modules, tests, etc.)

✓

```
.git
node_modules
__pycache__
*.md
.env
```

Buenas Prácticas (2/2)

Seguridad y mantenimiento

No ejecutar como root

Crear un usuario dedicado con RUN adduser -D appuser y luego USER appuser. Minimiza el impacto de una brecha de seguridad.

Fijar versiones en tags

Usar python:3.11.7-slim en vez de python:latest. Los tags :latest son mutables y pueden romper builds sin aviso.

Multi-stage builds

Compilar en una etapa y copiar solo el binario al runtime. Reduce el tamaño final drásticamente eliminando compiladores.

Escanear vulnerabilidades

Integrar Trivy o Docker Scout en el pipeline CI/CD. Escanear antes de cada push al registry para detectar CVEs.

Healthcheck y metadata

Incluir HEALTHCHECK para monitoreo y LABEL para trazabilidad (autor, versión, descripción).

Limpiar en la misma capa

Eliminar caches (apt, pip, npm) en el mismo RUN que los instala. Archivos borrados en otra capa siguen en la imagen.

Multi-Stage Build

Compilar en una etapa, ejecutar en otra — imágenes mínimas

Dockerfile (multi-stage)

```
# === STAGE 1: Build ==================
FROM golang:1.22-alpine AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app/server .

# === STAGE 2: Runtime ================
FROM alpine:3.19
RUN apk --no-cache add ca-certificates
COPY --from=builder /app/server /usr/local/bin/
RUN adduser -D appuser
USER appuser
EXPOSE 8080
CMD ["server"]
```

Resultado

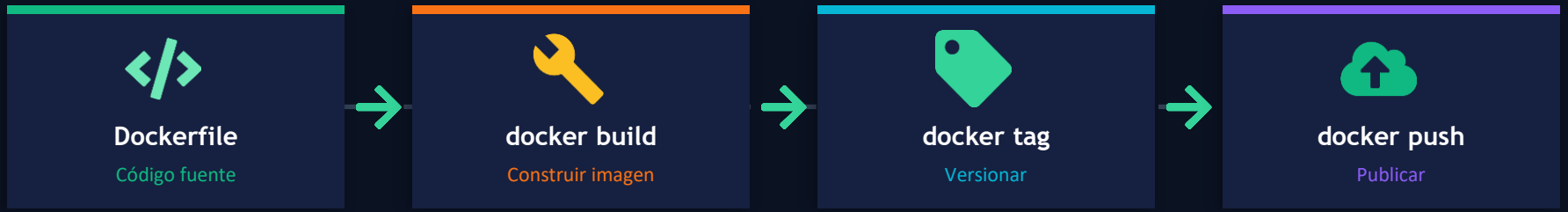
~1.2 GB	~15 MB
Build	Final

Solo se incluye en la imagen final:

- Binario compilado
- Certificados CA
- Alpine base (~7MB)
- Usuario no-root

Build, Tag y Push

Flujo completo: de Dockerfile a DockerHub




```
Terminal $
# 1. Construir la imagen con tag
docker build -t mi-app:1.0.0 .

# 2. Etiquetar para DockerHub (usuario/repo:tag)
docker tag mi-app:1.0.0 miusuario/mi-app:1.0.0
docker tag mi-app:1.0.0 miusuario/mi-app:latest

# 3. Autenticarse en DockerHub
docker login -u miusuario

# 4. Subir la imagen al registry
docker push miusuario/mi-app:1.0.0
docker push miusuario/mi-app:latest
```

Estrategia de Tags

Convenciones de versionado para imágenes Docker

Tag	Descripción	Riesgo
`latest`	Tag por defecto. Mutable, se sobreescribe con cada push. Nunca usar en producción.	Alto
`1.0.0`	Semantic Versioning (MAJOR.MINOR.PATCH). Inmutable y predecible. Recomendado para producción.	Bajo
`1.0`	Apunta al último patch de esa minor. Balance entre estabilidad y actualizaciones.	Medio
`sha-a3f4b2c`	Hash del commit Git. Máxima trazabilidad. Ideal para CI/CD y debugging.	Bajo
`alpine / slim`	Variante de la imagen base. Alpine (~5MB) vs slim (~50MB) vs full (~200MB+).	N/A

Recomendación: En producción, siempre usar tags inmutables (semver o SHA). Combinar: docker tag app:1.2.3 app:sha-a3f4b2c

DockerHub

El registry público más grande del ecosistema Docker

Registry Público

Más de 8 millones de imágenes disponibles. Imágenes oficiales verificadas por Docker Inc.

Imágenes Oficiales

Mantenidas por Docker. Actualizadas, escaneadas y documentadas. Ejemplo: nginx, postgres, python.

Automated Builds

Conectar repositorio Git (GitHub/Bitbucket). Build automático en cada push o merge a main.

Repositorios Privados

Plan gratuito: 1 repo privado. Planes de pago para equipos con repos ilimitados.

Vulnerability Scanning

Docker Scout analiza las imágenes detectando CVEs en dependencias y paquetes del SO.

Webhooks y CI/CD

Notificaciones automáticas al hacer push. Integración con Jenkins, GitHub Actions, GitLab CI.

Crear Cuenta en DockerHub

Paso a paso: registro, login CLI y configuración

1 Registrarse en hub.docker.com

1. Ir a https://hub.docker.com
2. Click en "Sign Up"
3. Ingresar: Docker ID (será tu usuario), email y contraseña
4. Verificar email
5. Elegir plan (Personal = gratuito)

2 Crear Repositorio

1. En DockerHub → "Create Repository"
2. Nombre: mi-app-lab
3. Descripción: Lab Unidad 2
4. Visibilidad: Public (gratis) o Private (1 gratis)
5. Click "Create"

3 Conectarse desde la Terminal

```
# Login interactivo (pide password)
docker login -u tu-docker-id

# Login con token (más seguro, recomendado para CI/CD)
# Crear token en: hub.docker.com > Account Settings > Security > New Access Token
echo 'dckr_pat_xxxxx' | docker login -u tu-docker-id --password-stdin

# Verificar login exitoso
cat ~/.docker/config.json   # Muestra credenciales almacenadas
```

Flujo Completo: Build → Tag → Push

De tu máquina local a DockerHub en 4 pasos

Terminal $ — Flujo completo paso a paso

```
# ═══ PASO 1: Construir la imagen ═══════════════════
cd ~/lab3-dockerhub
docker build -t mi-app-lab:1.0.0 -f Dockerfile.lab3-dockerhub .

# ═══ PASO 2: Verificar que se creó ═══════════════
docker images | grep mi-app-lab
# mi-app-lab   1.0.0   abc123def456   5 seconds ago   150MB

# ═══ PASO 3: Tag con tu usuario de DockerHub ═══
# Formato: docker tag <imagen_local> <dockerhub_user>/<repo>:<tag>
docker tag mi-app-lab:1.0.0 tu-usuario/mi-app-lab:1.0.0
docker tag mi-app-lab:1.0.0 tu-usuario/mi-app-lab:latest

# ═══ PASO 4: Push al registry ═══════════════════
docker push tu-usuario/mi-app-lab:1.0.0
docker push tu-usuario/mi-app-lab:latest

# ═══ VERIFICAR ═══════════════════════════════════
# Tu imagen está en: https://hub.docker.com/r/tu-usuario/mi-app-lab
# Cualquiera puede hacer: docker pull tu-usuario/mi-app-lab:1.0.0
```

Vulnerabilidades en Imágenes

Riesgos de seguridad en la cadena de suministro de contenedores

CVEs en Imagen Base

Las imágenes base pueden contener paquetes del SO con vulnerabilidades conocidas. Una imagen ubuntu:22.04 sin parchear puede tener 50+ CVEs.

Dependencias Inseguras

Librerías de la aplicación (pip, npm, maven) con vulnerabilidades. Supply chain attacks como el caso de event-stream.

Secretos Expuestos

API keys, contraseñas o tokens hardcodeados en el Dockerfile o copiados a la imagen. Visibles con docker history.

Ejecutar como Root

Contenedores corriendo como root. Si un atacante logra escape del contenedor, obtiene root en el host.

Imágenes Sin Verificar

Descargar imágenes de registries no oficiales o sin firma. Riesgo de imágenes con malware o backdoors.

Capas con Datos Sensibles

Aunque borres un archivo en una capa posterior, sigue existiendo en capas anteriores. Recuperable con dive.

Herramientas de Escaneo

Detectar vulnerabilidades antes de desplegar

Trivy

Open source (Aqua Security). Escanea imágenes, filesystems y repos. El más popular en CI/CD.

```
$ trivy image mi-app:1.0.0
```

Docker Scout

Integrado en Docker Desktop y CLI. SBOM y recomendaciones de remediación con fix versions.

```
$ docker scout cves mi-app:1.0.0
```

Grype

Open source (Anchore). Generador SBOM con syft + escáner de CVEs. Nativo con GitHub Actions.

```
$ grype mi-app:1.0.0
```

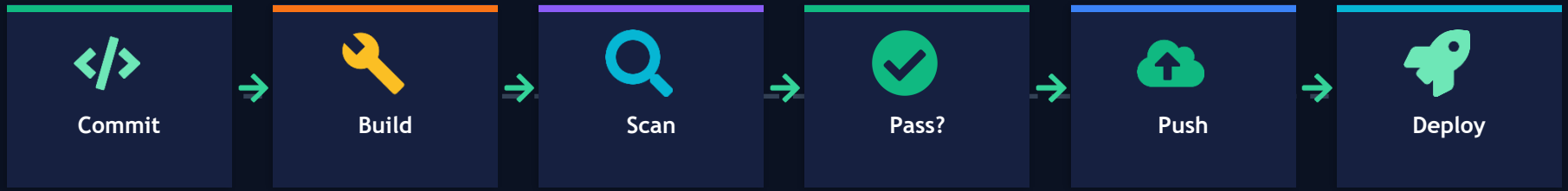
Snyk Container

Plataforma SaaS con tier gratuito. Monitoreo continuo, alertas por nuevos CVEs.

```
$ snyk container test mi-app:1.0.0
```

Pipeline CI/CD con Escaneo

Integrar escaneo de vulnerabilidades en el flujo de despliegue



GitHub Actions (.github/workflows/scan.yml)

```yaml
- name: Build Docker image
  run: docker build -t ${{ github.repository }}:${{ github.sha }} .

- name: Scan with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${{ github.repository }}:${{ github.sha }}
    severity: HIGH,CRITICAL
    exit-code: 1          # Falla el pipeline si hay CVEs

- name: Push to DockerHub
  if: success()           # Solo si el escaneo pasa
  run: docker push ${{ github.repository }}:${{ github.sha }}
```

Laboratorio Práctico

4 Laboratorios — Dockerfile, Build, DockerHub y Escaneo

01

Multi-Stage Build (Go)

`Dockerfile.lab1-multistage-go`

02

Node.js Optimizado

`Dockerfile.lab2-nodejs-optimizado`

03

DockerHub: Cuenta y Push

`Dockerfile.lab3-dockerhub-push`

04

Escaneo con Trivy

`Dockerfile.lab4-escaneo-*`

Lab 1: Multi-Stage Build con Go

Dockerfile.lab1-multistage-go — Imagen final de ~15MB

Dockerfile.lab1-multistage-go

```
FROM golang:1.22-alpine AS builder
WORKDIR /src
COPY main.go .
RUN CGO_ENABLED=0 go build \
    -ldflags='-s -w' -o /app/server .

FROM scratch
COPY --from=builder /app/server /server
EXPOSE 8080
CMD ["/server"]
```

Despliegue paso a paso

```
# Crear directorio
mkdir -p ~/lab1-multistage && cd ~/lab1-multistage

# Crear main.go (ver archivos del lab)

# Construir imagen
docker build -t go-server:v1 \
  -f Dockerfile.lab1-multistage-go .

# Ver tamaño (debe ser ~10-15MB)
docker images go-server:v1

# Ejecutar
docker run -d --name go-app \
  -p 8080:8080 go-server:v1

# Probar
curl http://localhost:8080

# Comparar tamaños
docker images | grep -E 'golang|go-server'
```

Lab 2: Crear Cuenta DockerHub + Push

Dockerfile.lab3-dockerhub-push — Registro, login y publicación

Guía completa paso a paso

```
# ═══ PASO 1: Crear cuenta en DockerHub ═══════════════
# Ir a https://hub.docker.com → Sign Up → Elegir plan Personal
# Crear Docker ID (ej: mi-usuario-lab) y verificar email

# ═══ PASO 2: Login desde terminal ═══════════════════
docker login -u mi-usuario-lab
# Ingresar password cuando lo pida

# ═══ PASO 3: Construir imagen del lab ═══════════════
cd ~/lab3-dockerhub
docker build -t flask-dockerhub:1.0.0 \
  -f Dockerfile.lab3-dockerhub-push .

# ═══ PASO 4: Tag con tu usuario ═══════════════════
docker tag flask-dockerhub:1.0.0 mi-usuario-lab/flask-dockerhub:1.0.0
docker tag flask-dockerhub:1.0.0 mi-usuario-lab/flask-dockerhub:latest

# ═══ PASO 5: Push a DockerHub ═══════════════════
docker push mi-usuario-lab/flask-dockerhub:1.0.0
docker push mi-usuario-lab/flask-dockerhub:latest

# ═══ PASO 6: Verificar en la web ═══════════════
# Ir a https://hub.docker.com/r/mi-usuario-lab/flask-dockerhub
# Verificar que aparecen los tags 1.0.0 y latest

# ═══ PASO 7: Pull desde otra máquina (o limpiar y bajar) =
docker rmi mi-usuario-lab/flask-dockerhub:1.0.0
docker pull mi-usuario-lab/flask-dockerhub:1.0.0
```

Lab 3: Escaneo de Vulnerabilidades

Dockerfile.lab4-escaneo-inseguro vs Dockerfile.lab4-escaneo-seguro

```
# ═══ DESCARGA DE IMAGEN═══════════════════════════
sagikazarmark/dvwa:latest

# ═══ ESCANEO DE VULNERABILIDADES ═══════════════════════════
docker run \
  -v trivy-cache:/root/.cache/trivy \
  aquasec/trivy image \
  --severity CRITICAL,HIGH \
  sagikazarmark/dvwa:latest 2>&1 | more
```

Comandos Útiles — Unidad 2

Referencia rápida para gestión de imágenes

Imágenes y Build

`docker build -t name:tag .`

Construir imagen desde Dockerfile

`docker build -f Dockerfile.lab1 .`

Build con Dockerfile específico

`docker build --no-cache .`

Build sin cache (fuerza rebuild)

`docker images`

Listar imágenes locales

`docker history <img>`

Ver capas y tamaño de cada una

`docker rmi <img>`

Eliminar imagen

Tags, Push y Registry

`docker tag img:v1 user/img:v1`

Etiquetar para un registry

`docker login`

Autenticar en DockerHub

`docker push user/img:v1`

Subir imagen al registry

`docker pull user/img:v1`

Descargar imagen del registry

`docker search nginx`

Buscar en DockerHub

`docker system prune -a`

Eliminar todo sin uso

Resumen — Unidad 2

- El Dockerfile define la imagen capa por capa — cada instrucción es determinista e inmutable
- Buenas prácticas: imágenes slim, cache de deps, multi-stage, non-root, .dockerignore
- Flujo Build → Tag (semver) → Push — nomenclatura con nombre de laboratorio en Dockerfile
- DockerHub: crear cuenta, login CLI, crear repositorio, push y pull de imágenes
- Escanear siempre: Trivy y Docker Scout detectan CVEs antes de llegar a producción